Tutorial: Getting started with generics

Table of Contents

Prerequisites
Create a folder for your code
Add non-generic functions
Add a generic function to handle multiple types
Remove type arguments when calling the generic function
Declare a type constraint
Conclusion
Completed code

This tutorial introduces the basics of generics in Go. With generics, you can declare and use functions or types that are written to work with any of a set of types provided by calling code.

In this tutorial, you'll declare two simple non-generic functions, then capture the same logic in a single generic function.

You'll progress through the following sections:

1. Create a folder for your code.
2. Add non-generic functions.
3. Add a generic function to handle multiple types.
4. Remove type arguments when calling the generic function.
5. Declare a type constraint.

Note: For other tutorials, see Tutorials.

Note: If you prefer, you can use the Go playground in "Go dev branch" mode to edit and run your program instead.

Prerequisites

- **An installation of Go 1.18 or later.** For installation instructions, see Installing Go.
- **A tool to edit your code.** Any text editor you have will work fine.
- **A command terminal.** Go works well using any terminal on Linux and Mac, and on PowerShell or cmd in Windows.

Create a folder for your code

To begin, create a folder for the code you'll write.

1. Open a command prompt and change to your home directory.

   On Linux or Mac:

   ```
   $ cd
   ```

   On Windows:

   ```
   C:\> cd %HOMEPATH%
   ```

   The rest of the tutorial will show a $ as the prompt. The commands you use will work on Windows too.

2. From the command prompt, create a directory for your code called generics.

   ```
   $ mkdir generics
   $ cd generics
   ```

3. Create a module to hold your code.

   Run the `go mod init` command, giving it your new code's module path.

   ```
   $ go mod init example/generics
   go: creating new go.mod: module example/generics
   ```

   Note: For production code, you'd specify a module path that's more specific to your own needs. For more, be sure to see Managing dependencies.

Next, you'll add some simple code to work with maps.

Add non-generic functions

In this step, you'll add two functions that each add together the values of a map and return the total.

You're declaring two functions instead of one because you're working with two different types of maps: one that stores `int64` values, and one that stores `float64` values.

Write the code

1. Using your text editor, create a file called main.go in the generics directory. You'll write your Go code in this file.

2. Into main.go, at the top of the file, paste the following package declaration.

   ```
   package main
   ```

   A standalone program (as opposed to a library) is always in package `main`.

3. Beneath the package declaration, paste the following two function declarations.

   ```
   // SumInts adds together the values of m.
   func SumInts(m map[string]int64) int64 {
       var s int64
       for _, v := range m {
           s += v
       }
       return s
   }

   // SumFloats adds together the values of m.
   func SumFloats(m map[string]float64) float64 {
       var s float64
       for _, v := range m {
           s += v
       }
       return s
   }
   ```

   In this code, you:

   - Declare two functions to add together the values of a map and return the sum.
     - `SumFloats` takes a map of `string` to `float64` values.
     - `SumInts` takes a map of `string` to `int64` values.

4. At the top of main.go, beneath the package declaration, paste the following `main` function to initialize the two maps and use them as arguments when calling the functions you declared in the preceding step.

   ```
   func main() {
       // Initialize a map for the integer values
       ints := map[string]int64{
           "first":  34,
           "second": 12,
       }

       // Initialize a map for the float values
       floats := map[string]float64{
           "first":  35.98,
           "second": 26.99,
       }

       fmt.Printf("Non-Generic Sums: %v and %v\n",
           SumInts(ints),
           SumFloats(floats))
   }
   ```

   In this code, you:

   - Initialize a map of `float64` values and a map of `int64` values, each with two entries.
   - Call the two functions you declared earlier to find the sum of each map's values.
   - Print the result.

5. Near the top of main.go, just beneath the package declaration, import the package you'll need to support the code you've just written.

   The first lines of code should look like this:

   ```
   package main

   import "fmt"
   ```

6. Save main.go.

Run the code

From the command line in the directory containing main.go, run the code.

```
$ go run .
Non-Generic Sums: 46 and 62.97
```

With generics, you can write one function here instead of two. Next, you'll add a single generic function for maps containing either integer or float values.

Add a generic function to handle multiple types

In this section, you'll add a single generic function that can receive a map containing either integer or float values, effectively replacing the two functions you just wrote with a single function.

To support values of either type, that single function will need a way to declare what types it supports. Calling code, on the other hand, will need a way to specify whether it is calling with an integer or float map.

To support this, you'll write a function that declares *type parameters* in addition to its ordinary function parameters. These type parameters make the function generic, enabling it to work with arguments of different types. You'll call the function with *type arguments* and ordinary function arguments.

Each type parameter has a *type constraint* that acts as a kind of meta-type for the type parameter. Each type constraint specifies the permissible type arguments that calling code can use for the respective type parameter.

While a type parameter's constraint typically represents a set of types, at compile time the type parameter stands for a single type – the type provided as a type argument by the calling code. If the type argument's type isn't allowed by the type parameter's constraint, the code won't compile.

Keep in mind that a type parameter must support all the operations the generic code is performing on it. For example, if your function's code were to try to perform `string` operations (such as indexing) on a type parameter whose constraint included numeric types, the code wouldn't compile.

In the code you're about to write, you'll use a constraint that allows either integer or float types.

Write the code

1. Beneath the two functions you added previously, paste the following generic function.

   ```
   // SumIntsOrFloats sums the values of map m. It supports both int64 and float64
   // as types for map values.
   func SumIntsOrFloats[K comparable, V int64 | float64](m map[K]V) V {
       var s V
       for _, v := range m {
           s += v
       }
       return s
   }
   ```

   In this code, you:

   - Declare a `SumIntsOrFloats` function with two type parameters (inside the square brackets), K and V, and one argument that uses the type parameters, m of type map[K]V. The function returns a value of type V.
   - Specify for the K type parameter the type constraint `comparable`. Intended specifically for cases like these, the `comparable` constraint is predeclared in Go. It allows any type whose values may be used as an operand of the comparison operators == and !=. Go requires that map keys be comparable. So declaring K as `comparable` is necessary so you can use K as the key in the map variable. It also ensures that calling code uses an allowable type for map keys.
   - Specify for the V type parameter a constraint that is a union of two types: `int64` and `float64`. Using `|` specifies a union of the two types, meaning that this constraint allows either type. Either type will be permitted by the compiler as an argument in the calling code.
   - Specify that the m argument is of type map[K]V, where K and V are the types already specified for the type parameters. Note that we know map[K]V is a valid map type because K is a comparable type. If we hadn't declared K comparable, the compiler would reject the reference to map[K]V.

2. In main.go, beneath the code you already have, paste the following code.

   ```
   fmt.Printf("Generic Sums: %v and %v\n",
       SumIntsOrFloats[string, int64](ints),
       SumIntsOrFloats[string, float64](floats))
   ```

   In this code, you:

   - Call the generic function you just declared, passing each of the maps you created.
   - Specify type arguments – the type names in square brackets – to be clear about the types that should replace type parameters in the function you're calling.

     As you'll see in the next section, you can often omit the type arguments in the function call. Go can often infer them from your code.

   - Print the sums returned by the function.

Run the code

From the command line in the directory containing main.go, run the code.

```
$ go run .
Non-Generic Sums: 46 and 62.97
Generic Sums: 46 and 62.97
```

To run your code, in each call the compiler replaced the type parameters with the concrete types specified in that call.

In calling the generic function you wrote, you specified type arguments that told the compiler what types to use in place of the function's type parameters. As you'll see in the next section, in many cases you can omit these type arguments because the compiler can infer them.

Remove type arguments when calling the generic function

In this section, you'll add a modified version of the generic function call, making a small change to simplify the calling code. You'll remove the type arguments, which aren't needed in this case.

You can omit type arguments in calling code when the Go compiler can infer the types you want to use. The compiler infers type arguments from the types of function arguments.

Note that this isn't always possible. For example, if you needed to call a generic function that had no arguments, you would need to include the type arguments in the function call.

Write the code

- In main.go, beneath the code you already have, paste the following code.

  ```
  fmt.Printf("Generic Sums, type parameters inferred: %v and %v\n",
      SumIntsOrFloats(ints),
      SumIntsOrFloats(floats))
  ```

  In this code, you:

  - Call the generic function, omitting the type arguments.

Run the code

From the command line in the directory containing main.go, run the code.

```
$ go run .
Non-Generic Sums: 46 and 62.97
Generic Sums: 46 and 62.97
Generic Sums, type parameters inferred: 46 and 62.97
```

Next, you'll further simplify the function by capturing the union of integers and floats into a type constraint you can reuse, such as from other code.

Declare a type constraint

In this last section, you'll move the constraint you defined earlier into its own interface so that you can reuse it in multiple places. Declaring constraints in this way helps streamline code, such as when a constraint is more complex.

You declare a *type constraint* as an interface. The constraint allows any type implementing the interface. For example, if you declare a type constraint interface with three methods, then use it with a type parameter in a generic function, type arguments used to call the function must have all of those methods.

Constraint interfaces can also refer to specific types, as you'll see in this section.

Write the code

1. Just above `main`, immediately after the import statements, paste the following code to declare a type constraint.

   ```
   type Number interface {
       int64 | float64
   }
   ```

   In this code, you:

   - Declare the `Number` interface type to use as a type constraint.
   - Declare a union of `int64` and `float64` inside the interface.

     Essentially, you're moving the union from the function declaration into a new type constraint. That way, when you want to constrain a type parameter to either `int64` or `float64`, you can use this `Number` type constraint instead of writing out `int64 | float64`.

2. Beneath the functions you already have, paste the following generic `SumNumbers` function.

   ```
   // SumNumbers sums the values of map m. It supports both integers
   // and floats as map values.
   func SumNumbers[K comparable, V Number](m map[K]V) V {
       var s V
       for _, v := range m {
           s += v
       }
       return s
   }
   ```

   In this code, you:

   - Declare a generic function with the same logic as the generic function you declared previously, but with the new interface type instead of the union as the type constraint. As before, you use the type parameters for the argument and return types.

3. In main.go, beneath the code you already have, paste the following code.

   ```
   fmt.Printf("Generic Sums with Constraint: %v and %v\n",
       SumNumbers(ints),
       SumNumbers(floats))
   ```

   In this code, you:

   - Call `SumNumbers` with each map, printing the sum from the values of each.

     As in the preceding section, you omit the type arguments (the type names in square brackets) in calls to the generic function. The Go compiler can infer the type argument from other arguments.

Run the code

From the command line in the directory containing main.go, run the code.

```
$ go run .
Non-Generic Sums: 46 and 62.97
Generic Sums: 46 and 62.97
Generic Sums, type parameters inferred: 46 and 62.97
Generic Sums with Constraint: 46 and 62.97
```

Conclusion

Nicely done! You've just introduced yourself to generics in Go.

Suggested next topics:

- The Go Tour is a great step-by-step introduction to Go fundamentals.
- You'll find useful Go best practices described in Effective Go and How to write Go code.

Completed code

You can run this program in the Go playground. On the playground simply click the **Run** button.

```
package main

import "fmt"

type Number interface {
    int64 | float64
}

func main() {
    // Initialize a map for the integer values
    ints := map[string]int64{
        "first": 34,
        "second": 12,
    }

    // Initialize a map for the float values
    floats := map[string]float64{
        "first": 35.98,
        "second": 26.99,
    }

    fmt.Printf("Non-Generic Sums: %v and %v\n",
        SumInts(ints),
        SumFloats(floats))

    fmt.Printf("Generic Sums: %v and %v\n",
        SumIntsOrFloats[string, int64](ints),
        SumIntsOrFloats[string, float64](floats))

    fmt.Printf("Generic Sums, type parameters inferred: %v and %v\n",
        SumIntsOrFloats(ints),
        SumIntsOrFloats(floats))

    fmt.Printf("Generic Sums with Constraint: %v and %v\n",
        SumNumbers(ints),
        SumNumbers(floats))
}

// SumInts adds together the values of m.
func SumInts(m map[string]int64) int64 {
    var s int64
    for _, v := range m {
        s += v
    }
    return s
}

// SumFloats adds together the values of m.
func SumFloats(m map[string]float64) float64 {
    var s float64
    for _, v := range m {
        s += v
    }
    return s
}

// SumIntsOrFloats sums the values of map m. It supports both floats and integers
// as map values.
func SumIntsOrFloats[K comparable, V int64 | float64](m map[K]V) V {
    var s V
    for _, v := range m {
        s += v
    }
    return s
}

// SumNumbers sums the values of map m. Its supports both integers
// and floats as map values.
func SumNumbers[K comparable, V Number](m map[K]V) V {
    var s V
    for _, v := range m {
        s += v
    }
    return s
}
```